Volume I Mathematical Logic And Proof Reference

Contents

Propositional Logic

0.1 Model-Theoretic View

0.2 Axiom Schemas

0.3 Notation and Conventions

0.4 Syntax

Definition 0.4.1 (Truth Value)**.** The set of truth values is

$$\mathbb{B} := \{\text{False}, \text{True}\}.$$

Some sources identify this set with $\{0, 1\}$, with 0 read as false and 1 read as true.

Definition 0.4.2 (Propositional Language)**.** A propositional language consists of:

1. a set Prop of propositional variables;
2. logical connectives;
3. punctuation symbols such as parentheses;
4. recursive formation rules specifying the well-formed formulas.

Definition 0.4.3 (Propositional Variable)**.** A propositional variable is an atomic symbol intended to stand for a proposition. A standard supply of variables is

$$\mathsf{Prop} = \{P_1, P_2, P_3, \ldots\}.$$

Informally, variables are often written $P, Q, R, S, \ldots$.

Definition 0.4.4 (Logical Connective)**.** The standard primitive propositional connectives are

$$\neg, \qquad \wedge, \qquad \vee, \qquad \to, \qquad \leftrightarrow .$$

The connective $\neg$ is unary. The connectives $\wedge, \vee, \to, \leftrightarrow$ are binary.

Symbol	Name	Arity	Formation pattern	Reading
$\neg$	negation	unary	$\neg\varphi$	not φ
$\wedge$	conjunction	binary	$(\varphi \wedge \psi)$	φ and ψ
$\vee$	disjunction	binary	$(\varphi \vee \psi)$	φ or ψ
$\to$	conditional	binary	$(\varphi \to \psi)$	if φ, then ψ
$\leftrightarrow$	biconditional	binary	$(\varphi \leftrightarrow \psi)$	φ iff ψ

Definition 0.4.5 (Well-Formed Formula)**.** The set WFF of well-formed formulas is the smallest set of strings satisfying the following formation rules:

1. Every propositional variable $P \in \mathsf{Prop}$ belongs to WFF.
2. If $\varphi \in \mathsf{WFF}$, then $\neg\varphi \in \mathsf{WFF}$.
3. If $\varphi, \psi \in \mathsf{WFF}$ and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi) \in \mathsf{WFF}$.
4. No string is a well-formed formula unless it is obtained by finitely many applications of the previous clauses.

Lemma 0.4.6 (Closure Under Formation)**.** *The set* WFF *is closed under the propositional formation rules. That is, if* $\varphi \in \mathsf{WFF}$*, then*

$$\neg\varphi \in \mathsf{WFF}.$$

If $\varphi, \psi \in \mathsf{WFF}$ *and*

$$\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\},$$

then

$$(\varphi \circ \psi) \in \mathsf{WFF}.$$

Go to proof.

Theorem 0.4.7 (Minimality of Well-Formed Formulas)**.** *Let* S *be a set of strings over the propositional alphabet. Suppose:*

1. $\mathsf{Prop} \subseteq S$*;*

2. if $\varphi \in S$*, then* $\neg\varphi \in S$*;*

3. if $\varphi, \psi \in S$ *and* $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$*, then* $(\varphi \circ \psi) \in S$*.*

Then

$$\mathsf{WFF} \subseteq S.$$

Go to proof.

Definition 0.4.8 (Atomic Formula)**.** An atomic formula is a well-formed formula that is a propositional variable.

Definition 0.4.9 (Molecular Formula)**.** A molecular formula is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula formed using at least one logical connective.

Definition 0.4.10 (Main Connective)**.** The main connective of a molecular formula is the connective introduced at the final formation step:

1. for $\neg\varphi$, the main connective is $\neg$;
2. for $(\varphi \circ \psi)$, the main connective is $\circ$.

Atomic formulas have no main connective.

Theorem 0.4.11 (Structural Induction for Propositional Formulas)**.** *Let* $A(\varphi)$ *be a property of well-formed formulas. Suppose:*

1. $A(P)$ *holds for every* $P \in \mathsf{Prop}$*;*

2. if $A(\varphi)$ *holds, then* $A(\neg\varphi)$ *holds;*

3. if $A(\varphi)$ *and* $A(\psi)$ *hold, then* $A((\varphi \circ \psi))$ *holds for every* $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$*.*

Then $A(\theta)$ *holds for every* $\theta \in \mathsf{WFF}$*.*
Go to proof.

Theorem 0.4.12 (Structural Recursion for Propositional Formulas)**.** *To define a function F on WFF, it suffices to specify:*

1. *$F(P)$ for every $P \in \mathsf{Prop}$;*
2. *$F(\neg\varphi)$ in terms of $F(\varphi)$;*
3. *$F((\varphi \circ \psi))$ in terms of $F(\varphi)$, $F(\psi)$, and $\circ$.*

When these clauses respect the formation rules, they determine a unique function on WFF.
Go to proof.

Lemma 0.4.13 (Constructor Disjointness for Propositional Formulas)**.** *The three outer formation cases for propositional formulas are disjoint:*

1. *no propositional variable is a negation formula;*
2. *no propositional variable is a binary formula;*
3. *no negation formula is a binary formula.*

Go to proof.

Lemma 0.4.14 (Constructor Injectivity for Propositional Formulas)**.** *The propositional formula constructors are injective:*

1. *if $\neg\varphi = \neg\psi$, then $\varphi = \psi$;*
2. *if $(\varphi \circ \psi) = (\alpha \star \beta)$, then $\varphi = \alpha$, $\psi = \beta$, and $\circ = \star$.*

Go to proof.

Theorem 0.4.15 (Unique Decomposition for Propositional Formulas)**.** *Every $\varphi \in \mathsf{WFF}$ has exactly one outermost form:*

1. *$\varphi = P$ for a unique $P \in \mathsf{Prop}$;*
2. *$\varphi = \neg\psi$ for a unique $\psi \in \mathsf{WFF}$;*
3. *$\varphi = (\psi \circ \chi)$ for unique $\psi, \chi \in \mathsf{WFF}$ and a unique binary connective $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$.*

Go to proof.

Definition 0.4.16 (Parse Tree)**.** The parse tree, or syntax tree, of a well-formed formula is the tree representation of its recursive construction:

1. a propositional variable is represented by a single leaf;
2. a formula $\neg\varphi$ is represented by a root labeled $\neg$ with one child, the parse tree of φ;
3. a formula $(\varphi \circ \psi)$ is represented by a root labeled $\circ$ with two children, the parse trees of φ and ψ.

Theorem 0.4.17 (Unique Syntax Tree for Propositional Formulas)**.** *Every propositional well-formed formula has a unique parse tree.*
Go to proof.

Definition 0.4.18 (Variable Set of a Formula)**.** The variable set $\mathrm{Var}(\varphi)$ of a formula φ is defined recursively:

1. if $\varphi = P \in \mathsf{Prop}$, then $\mathrm{Var}(\varphi) = \{P\}$;
2. if $\varphi = \neg\psi$, then $\mathrm{Var}(\varphi) = \mathrm{Var}(\psi)$;

3. if $\varphi = (\psi \circ \chi)$, then
$$\mathrm{Var}(\varphi) = \mathrm{Var}(\psi) \cup \mathrm{Var}(\chi).$$

Definition 0.4.19 (Subformula)**.** The set $\mathrm{Sub}(\varphi)$ of subformulas of a formula φ is defined recursively:

1. if φ is atomic, then $\mathrm{Sub}(\varphi) = \{\varphi\}$;
2. if $\varphi = \neg\psi$, then
$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi);$$
3. if $\varphi = (\psi \circ \chi)$, then
$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi) \cup \mathrm{Sub}(\chi).$$

Definition 0.4.20 (Proper Subformula)**.** Let $\varphi \in \mathsf{WFF}$. A *proper subformula* of φ is a formula ψ such that
$$\psi \in \mathrm{Sub}(\varphi) \quad \text{and} \quad \psi \neq \varphi.$$

Definition 0.4.21 (Formula Depth)**.** The depth of a formula φ, denoted $\mathrm{depth}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\mathrm{depth}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\mathrm{depth}(\varphi) = \mathrm{depth}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then
$$\mathrm{depth}(\varphi) = \max\{\mathrm{depth}(\psi), \mathrm{depth}(\chi)\} + 1.$$

Definition 0.4.22 (Connective Count)**.** The connective count of a formula φ, denoted $\mathrm{conn}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\mathrm{conn}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\mathrm{conn}(\varphi) = \mathrm{conn}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then
$$\mathrm{conn}(\varphi) = \mathrm{conn}(\psi) + \mathrm{conn}(\chi) + 1.$$

Lemma 0.4.23 (Proper Subformula Depth)**.** *If ψ is a proper subformula of φ, then*
$$\mathrm{depth}(\psi) < \mathrm{depth}(\varphi).$$

Go to proof.

Lemma 0.4.24 (Finiteness of Variables in a Formula)**.** *For every $\varphi \in \mathsf{WFF}$, the set $\mathrm{Var}(\varphi)$ is finite.*
Go to proof.

Lemma 0.4.25 (Finiteness of Subformulas)**.** *For every $\varphi \in \mathsf{WFF}$, the set $\mathrm{Sub}(\varphi)$ is finite.*
Go to proof.

0.5 Semantics

Truth Assignments

Definition 0.5.1 (Truth Assignment)**.** A *truth assignment* is a function
$$v : \mathsf{Prop} \to \mathbb{B}.$$

For each propositional variable $P \in \mathsf{Prop}$, the value $v(P)$ is the truth value assigned to P.

Definition 0.5.2 (Domain of a Truth Assignment)**.** The *domain* of a truth assignment
$$v : \mathsf{Prop} \to \mathbb{B}$$

is the set Prop of propositional variables.

Extension of a Truth Assignment

Definition 0.5.3 (Extension of a Truth Assignment to Formulas)**.** Let

$$v : \mathsf{Prop} \to \mathbb{B}$$

be a truth assignment. The *extension* of v to well-formed formulas is the function

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

defined recursively as follows:

$$\widehat{v}(P) = v(P) \quad \text{for every } P \in \mathsf{Prop},$$

$$\widehat{v}(\neg\varphi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{False},$$

$$\widehat{v}(\varphi \wedge \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{True},$$

$$\widehat{v}(\varphi \vee \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ or } \widehat{v}(\psi) = \mathsf{True},$$

$$\widehat{v}(\varphi \to \psi) = \mathsf{False} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{False},$$

and

$$\widehat{v}(\varphi \leftrightarrow \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \widehat{v}(\psi).$$

Theorem 0.5.4 (Unique Extension of a Truth Assignment)**.** *Every truth assignment*

$$v : \mathsf{Prop} \to \mathbb{B}$$

has a unique extension

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables and the primitive connectives.
Go to proof.

Definition 0.5.5 (Satisfaction of a Formula)**.** Let v be a truth assignment and let $\varphi \in \mathsf{WFF}$. We say that v satisfies φ, written

$$v \models \varphi,$$

if

$$\widehat{v}(\varphi) = \text{True}.$$

Definition 0.5.6 (Satisfaction of a Set of Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. A truth assignment v satisfies Γ, written

$$v \models \Gamma,$$

if $v \models \gamma$ for every $\gamma \in \Gamma$.

Definition 0.5.7 (Truth Table)**.** Let $\varphi \in \mathsf{WFF}$ and suppose the variables occurring in φ are among $P_1, \ldots, P_n$. A truth table for φ is a finite table with one row for each assignment

$$a : \{P_1, \ldots, P_n\} \to \mathbb{B}$$

and a final column recording the value of $\widehat{v}(\varphi)$ for any truth assignment v extending a.

Definition 0.5.8 (Tautology)**.** A formula $\varphi \in \mathsf{WFF}$ is a tautology, written

$$\models \varphi,$$

if every truth assignment satisfies φ.

Definition 0.5.9 (Contradiction)**.** A formula $\varphi \in \mathsf{WFF}$ is a contradiction if no truth assignment satisfies φ.

Definition 0.5.10 (Satisfiable Formula)**.** A formula $\varphi \in \mathsf{WFF}$ is satisfiable if at least one truth assignment satisfies it.

Definition 0.5.11 (Contingency)**.** A formula $\varphi \in \mathsf{WFF}$ is a contingency if it is satisfiable but not a tautology.

Definition 0.5.12 (Logical Consequence)**.** Let $\Gamma \subseteq \mathsf{WFF}$ and $\varphi \in \mathsf{WFF}$. The formula φ is a logical consequence of Γ, written

$$\Gamma \models \varphi,$$

if every truth assignment satisfying Γ also satisfies φ.

Definition 0.5.13 (Logical Equivalence)**.** Formulas $\varphi, \psi \in \mathsf{WFF}$ are logically equivalent, written

$$\varphi \equiv \psi,$$

if they have the same truth value under every truth assignment.

Lemma 0.5.14 (Coincidence Lemma for Propositional Logic)**.** *Let $v, w : \mathsf{Prop} \to \mathbb{B}$ be truth assignments. If v and w agree on every propositional variable occurring in φ, then they assign the same truth value to φ:*

$$v|_{\mathrm{Var}(\varphi)} = w|_{\mathrm{Var}(\varphi)} \Longrightarrow \widehat{v}(\varphi) = \widehat{w}(\varphi).$$

Go to proof.

Theorem 0.5.15 (Finite Truth-Table Theorem)**.** *Every propositional formula has a finite truth table. More precisely, if* $\mathrm{Var}(\varphi)$ *has n elements, then φ has a truth table with 2^n rows.*
Go to proof.

Definition 0.5.16 (Unary Boolean Function)**.** A unary Boolean function is a function

$$f : \mathbb{B} \to \mathbb{B}.$$

Definition 0.5.17 (Binary Boolean Function)**.** A binary Boolean function is a function

$$f : \mathbb{B}^2 \to \mathbb{B},$$

equivalently a function

$$f : \mathbb{B} \times \mathbb{B} \to \mathbb{B}.$$

Definition 0.5.18 (Boolean Function)**.** An n-ary Boolean function is a function

$$f : \mathbb{B}^n \to \mathbb{B}.$$

Definition 0.5.19 (Truth Function Induced by a Formula)**.** Let $\varphi \in \mathsf{WFF}$, and suppose $\mathrm{Var}(\varphi) \subseteq \{P_1, \ldots, P_n\}$. The truth function induced by φ is the Boolean function

$$f_\varphi : \mathbb{B}^n \to \mathbb{B}$$

defined by

$$f_\varphi(b_1, \ldots, b_n) = \widehat{v}(\varphi),$$

where $v(P_i) = b_i$ for each i.

Theorem 0.5.20 (Counting Boolean Functions)**.** *There are exactly*

$$2^{2^n}$$

Boolean functions $\mathbb{B}^n \to \mathbb{B}$.
Go to proof.

0.6 Algebra of Propositions

Definition 0.6.1 (Propositional Equivalence Law)**.** A propositional equivalence law is a valid schema of the form
$$\varphi \equiv \psi,$$
with $\varphi, \psi \in \mathsf{WFF}$. Each instance of the schema says that the two formulas have the same truth value under every truth assignment.

Theorem 0.6.2 (Logical Equivalence is an Equivalence Relation)**.** *Logical equivalence is reflexive, symmetric, and transitive on* WFF*:*
$$\varphi \equiv \varphi, \qquad \varphi \equiv \psi \Longrightarrow \psi \equiv \varphi,$$
and
$$(\varphi \equiv \psi \land \psi \equiv \chi) \Longrightarrow \varphi \equiv \chi.$$

Go to proof.

Theorem 0.6.3 (Basic Boolean Equivalence Laws)**.** *For all formulas* $\varphi, \psi, \chi \in \mathsf{WFF}$*, the following logical equivalences hold.*
Commutativity:
$$\varphi \land \psi \equiv \psi \land \varphi, \qquad \varphi \lor \psi \equiv \psi \lor \varphi.$$
Associativity:
$$(\varphi \land \psi) \land \chi \equiv \varphi \land (\psi \land \chi), \qquad (\varphi \lor \psi) \lor \chi \equiv \varphi \lor (\psi \lor \chi).$$
Idempotence:
$$\varphi \land \varphi \equiv \varphi, \qquad \varphi \lor \varphi \equiv \varphi.$$
Double negation:
$$\neg\neg\varphi \equiv \varphi.$$

Go to proof.

Theorem 0.6.4 (Identity and Domination Laws)**.** *Let* $\top$ *be any tautological formula and let* $\bot$ *be any contradictory formula. Then for every* $\varphi \in \mathsf{WFF}$*:*
$$\varphi \land \top \equiv \varphi, \qquad \varphi \lor \bot \equiv \varphi,$$
and
$$\varphi \land \bot \equiv \bot, \qquad \varphi \lor \top \equiv \top.$$

Go to proof.

Theorem 0.6.5 (De Morgan Laws for Propositional Logic)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*
$$\neg(\varphi \land \psi) \equiv \neg\varphi \lor \neg\psi,$$
and
$$\neg(\varphi \lor \psi) \equiv \neg\varphi \land \neg\psi.$$

Go to proof.

Theorem 0.6.6 (Distributive Laws for Propositional Logic)**.** *For all* $\varphi, \psi, \chi \in \mathsf{WFF}$*,*
$$\varphi \land (\psi \lor \chi) \equiv (\varphi \land \psi) \lor (\varphi \land \chi),$$
and
$$\varphi \lor (\psi \land \chi) \equiv (\varphi \lor \psi) \land (\varphi \lor \chi).$$

Go to proof.

Theorem 0.6.7 (Absorption Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\varphi \vee \psi) \equiv \varphi,$$

and

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi.$$

Go to proof.

Theorem 0.6.8 (Conditional Equivalence Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$

and

$$\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi.$$

Go to proof.

Theorem 0.6.9 (Biconditional Equivalence Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Go to proof.

Definition 0.6.10 (Formula Context)**.** A formula context $C[-]$ is a well-formed formula with one distinguished placeholder occurrence $[-]$. For each well-formed formula φ, the expression $C[\varphi]$ denotes the formula obtained by replacing the placeholder with φ.

Theorem 0.6.11 (Replacement of Logical Equivalents)**.** *Replacing logically equivalent formulas inside a formula context preserves logical equivalence:*

$$\varphi \equiv \psi \Longrightarrow C[\varphi] \equiv C[\psi].$$

Go to proof.

0.7 Normal Forms

Orientation

Definition 0.7.1 (Literal)**.** A *literal* is either a propositional variable P or the negation $\neg P$ of a propositional variable P.

Definition 0.7.2 (Positive Literal)**.** A *positive literal* is a literal of the form P, where P is a propositional variable.

Definition 0.7.3 (Negative Literal)**.** A *negative literal* is a literal of the form $\neg P$, where P is a propositional variable.

Definition 0.7.4 (Complementary Literals)**.** Two literals are *complementary* if they are P and $\neg P$ for the same propositional variable P.

Definition 0.7.5 (Clause)**.** A *clause* is a finite disjunction of literals.

Definition 0.7.6 (Term)**.** A *term*, also called a *conjunction term*, is a finite conjunction of literals.

Definition 0.7.7 (Empty Clause)**.** The *empty clause* is the empty disjunction of literals. Semantically, it is read as false.

Definition 0.7.8 (Empty Conjunction)**.** The *empty conjunction* is the conjunction over no literals. Semantically, it is read as true.

Definition 0.7.9 (Negation Normal Form)**.** A formula is in *negation normal form* if it uses only propositional variables, negated propositional variables, conjunction, and disjunction, and every negation occurs immediately before a propositional variable.

Definition 0.7.10 (NNF Conversion Procedure)**.** The *NNF conversion procedure* converts a propositional formula to an equivalent formula in negation normal form by the following steps:

1. eliminate biconditionals using biconditional equivalence laws;
2. eliminate conditionals using conditional equivalence laws;
3. eliminate double negations;
4. push negations inward using De Morgan laws.

Theorem 0.7.11 (Existence of Negation Normal Form)**.** *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in negation normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Definition 0.7.12 (Elementary Conjunction)**.** An *elementary conjunction* is a finite conjunction of literals. That is, it is a term of the form

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Definition 0.7.13 (Minterm)**.** Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *minterm* relative to $P_1, \ldots, P_n$ is an elementary conjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Definition 0.7.14 (Disjunctive Normal Form)**.** A formula is in *disjunctive normal form*, abbreviated *DNF*, when it is a finite disjunction of elementary conjunctions. Equivalently, it has the shape

$$C_1 \vee C_2 \vee \cdots \vee C_m,$$

where each C_j is an elementary conjunction.

Theorem 0.7.15 (Existence of Disjunctive Normal Form)**.** *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in disjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Theorem 0.7.16 (DNF Satisfiability Criterion)**.** *Let ψ be a formula in disjunctive normal form. Then ψ is satisfiable if and only if at least one conjunction term in ψ contains no complementary pair of literals.* *Go to proof.*

Definition 0.7.17 (Elementary Disjunction)**.** An *elementary disjunction* is a finite disjunction of literals. That is, it is a clause of the form

$$L_1 \vee L_2 \vee \cdots \vee L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Definition 0.7.18 (Maxterm)**.** Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *maxterm* relative to $P_1, \ldots, P_n$ is an elementary disjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Definition 0.7.19 (Conjunctive Normal Form)**.** A formula is in *conjunctive normal form*, abbreviated *CNF*, when it is a finite conjunction of clauses. Equivalently, it has the shape

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each C_j is a clause.

Theorem 0.7.20 (Existence of Conjunctive Normal Form)**.** *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in conjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Definition 0.7.21 (Truth-Table Row Conjunction)**.** Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row conjunction* for r is the conjunction

$$L_1 \wedge L_2 \wedge \cdots \wedge L_n,$$

where

$$L_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ true,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ false.} \end{cases}$$

Definition 0.7.22 (Truth-Table Row Clause)**.** Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row clause* for r is the clause

$$M_1 \vee M_2 \vee \cdots \vee M_n,$$

where

$$M_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ false,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ true.} \end{cases}$$

Definition 0.7.23 (Canonical Disjunctive Normal Form)**.** Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical disjunctive normal form* of φ relative to this ordering is the disjunction of the truth-table row conjunctions corresponding to the rows on which φ is true.

Definition 0.7.24 (Canonical Conjunctive Normal Form)**.** Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical conjunctive normal form* of φ relative to this ordering is the conjunction of the truth-table row clauses corresponding to the rows on which φ is false.

Theorem 0.7.25 (Canonical Disjunctive Normal Form Theorem)**.** *Every formula $\varphi \in \mathsf{WFF}$ is logically equivalent to its canonical disjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CDNF}_{P_1,\dots,P_n}(\varphi).$$

Go to proof.

Theorem 0.7.26 (Canonical Conjunctive Normal Form Theorem)**.** *Every formula $\varphi \in \mathsf{WFF}$ is logically equivalent to its canonical conjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CCNF}_{P_1,\dots,P_n}(\varphi).$$

Go to proof.

Theorem 0.7.27 (Semantic Classification by Normal Forms)**.** *Normal forms give finite certificates for semantic classification. For any formula $\varphi \in \mathsf{WFF}$:*

1. *φ is satisfiable if and only if some DNF formula equivalent to φ has a satisfiable conjunction term.*
2. *φ is a contradiction if and only if some DNF formula equivalent to φ has no satisfiable conjunction term.*
3. *φ is a tautology if and only if some CNF formula equivalent to φ has no falsifiable clause.*
4. *φ is a contingency if and only if it is satisfiable and not a tautology.*

Go to proof.

Summary

0.8 Functional Completeness

Orientation

Boolean Functions and Representation

Definition 0.8.1 (Representation of a Boolean Function)**.** Let $f : \mathbb{B}^n \to \mathbb{B}$ be a Boolean function. A formula $\varphi \in \mathsf{WFF}$ with variables among $P_1, \ldots, P_n$ *represents* f if for every truth assignment v,

$$\widehat{v}(\varphi) = f(v(P_1), \ldots, v(P_n)).$$

Theorem 0.8.2 (DNF Representation of Boolean Functions)**.** *Every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is represented by a formula in disjunctive normal form using variables $P_1, \ldots, P_n$.*
Go to proof.

Connective Bases

Definition 0.8.3 (Connective Basis)**.** A *connective basis* is a specified set B of propositional connectives allowed as primitive connectives for building formulas.

Definition 0.8.4 (Definable Connective)**.** Let B be a connective basis. A connective $\circ$ is *definable from B* if there is a formula built using only connectives from B whose truth function agrees with the truth function of $\circ$.

Definition 0.8.5 (Functionally Complete Connective Set)**.** A connective basis B is *functionally complete* if every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$, for every $n \geq 1$, is represented by some formula using only connectives from B.

Definition 0.8.6 (Expressively Equivalent Connective Sets)**.** Two connective bases B and C are *expressively equivalent* if every connective in B is definable from C, and every connective in C is definable from B.

Completeness of the Standard Connectives

Theorem 0.8.7 (Standard Connectives are Functionally Complete)**.** *The connective basis*

$$\{\neg, \wedge, \vee\}$$

is functionally complete.
Go to proof.

Theorem 0.8.8 (Negation and Conjunction are Functionally Complete)**.** *The connective basis*

$$\{\neg, \wedge\}$$

is functionally complete.
Go to proof.

Theorem 0.8.9 (Negation and Disjunction are Functionally Complete)**.** *The connective basis*

$$\{\neg, \vee\}$$

is functionally complete.
Go to proof.

Reduction of Connective Sets

Definition 0.8.10 (Connective Reduction)**.** A *connective reduction* replaces a formula using one connective basis by a logically equivalent formula using a smaller or different connective basis.

NAND

Definition 0.8.11 (NAND Connective). The *NAND* connective, written $\uparrow$, is the binary connective defined by

$$\varphi \uparrow \psi \equiv \neg(\varphi \wedge \psi).$$

Definition 0.8.12 (NAND-Only Formula). A *NAND-only formula* is a well-formed formula whose only connective is $\uparrow$.

Theorem 0.8.13 (NAND is Functionally Complete). *The one-connective basis $\{\uparrow\}$ is functionally complete.*
Go to proof.

NOR

Definition 0.8.14 (NOR Connective). The *NOR* connective, written $\downarrow$, is the binary connective defined by

$$\varphi \downarrow \psi \equiv \neg(\varphi \vee \psi).$$

Definition 0.8.15 (NOR-Only Formula). A *NOR-only formula* is a well-formed formula whose only connective is $\downarrow$.

Theorem 0.8.16 (NOR is Functionally Complete). *The one-connective basis $\{\downarrow\}$ is functionally complete.*
Go to proof.

Incomplete Connective Sets

Definition 0.8.17 (Incomplete Connective Set). A connective basis is *incomplete* if it is not functionally complete.

Definition 0.8.18 (Monotone Boolean Function). A Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is *monotone* if changing input coordinates from False to True never changes the output from True to False.

Theorem 0.8.19 (Conjunction and Disjunction are not Functionally Complete). *The connective basis $\{\wedge, \vee\}$ is not functionally complete.*
Go to proof.

Minimal Complete Bases

Definition 0.8.20 (Minimal Complete Basis). A functionally complete connective basis B is *minimal complete* if no proper subset of B is functionally complete.

Theorem 0.8.21 (NAND and NOR are Minimal Complete Bases). *The one-connective bases $\{\uparrow\}$ and $\{\downarrow\}$ are minimal complete bases.*
Go to proof.

Summary and Transition

0.9 Argument Forms and Informal Proof Patterns

Orientation

Argument Forms

Definition 0.9.1 (Argument Form). An *argument form* is a schematic pattern consisting of a finite list of premise formulas and one conclusion formula.

Definition 0.9.2 (Valid Argument Form)**.** An argument form with premises Γ and conclusion φ is *valid* if every truth assignment that satisfies every formula in Γ also satisfies φ.

Definition 0.9.3 (Counterexample Assignment)**.** A *counterexample assignment* to an argument form (Γ, φ) is a truth assignment that satisfies every premise in Γ and does not satisfy φ.

Theorem 0.9.4 (Truth-Table Validity Test for Argument Forms)**.** *An argument form* (Γ, φ) *is valid if and only if no row of its truth table makes every formula in* Γ *true and* φ *false.*
Go to proof.

Core Valid Inference Patterns

Theorem 0.9.5 (Modus Ponens is Valid)**.** *The argument form*

$$\varphi, \qquad \varphi \to \psi \quad \therefore \quad \psi$$

is valid.
Go to proof.

Theorem 0.9.6 (Modus Tollens is Valid)**.** *The argument form*

$$\varphi \to \psi, \qquad \neg\psi \quad \therefore \quad \neg\varphi$$

is valid.
Go to proof.

Theorem 0.9.7 (Hypothetical Syllogism is Valid)**.** *The argument form*

$$\varphi \to \psi, \qquad \psi \to \chi \quad \therefore \quad \varphi \to \chi$$

is valid.
Go to proof.

Theorem 0.9.8 (Disjunctive Syllogism is Valid)**.** *The argument form*

$$\varphi \vee \psi, \qquad \neg\varphi \quad \therefore \quad \psi$$

is valid.
Go to proof.

Theorem 0.9.9 (Addition is Valid)**.** *The argument form*

$$\varphi \quad \therefore \quad \varphi \vee \psi$$

is valid.
Go to proof.

Theorem 0.9.10 (Simplification is Valid)**.** *The argument form*

$$\varphi \wedge \psi \quad \therefore \quad \varphi$$

is valid.
Go to proof.

Theorem 0.9.11 (Conjunction Introduction is Valid)**.** *The argument form*

$$\varphi, \qquad \psi \quad \therefore \quad \varphi \wedge \psi$$

is valid.
Go to proof.

Theorem 0.9.12 (Constructive Dilemma is Valid)**.** *The argument form*

$$\varphi \to \chi, \qquad \psi \to \theta, \qquad \varphi \vee \psi \quad \therefore \quad \chi \vee \theta$$

is valid.
Go to proof.

Theorem 0.9.13 (Destructive Dilemma is Valid)**.** *The argument form*

$$\varphi \to \chi, \qquad \psi \to \theta, \qquad \neg\chi \vee \neg\theta \quad \therefore \quad \neg\varphi \vee \neg\psi$$

is valid.
Go to proof.

Invalid Argument Forms

Theorem 0.9.14 (Affirming the Consequent is Invalid)**.** *The argument form*

$$\varphi \to \psi, \qquad \psi \quad \therefore \quad \varphi$$

is invalid.
Go to proof.

Theorem 0.9.15 (Denying the Antecedent is Invalid)**.** *The argument form*

$$\varphi \to \psi, \qquad \neg\varphi \quad \therefore \quad \neg\psi$$

is invalid.
Go to proof.

Rules of Replacement

Definition 0.9.16 (Rule of Replacement)**.** A *rule of replacement* is a schematic permission to replace a subformula by a logically equivalent formula inside a formula context.

Theorem 0.9.17 (Replacement Rules Preserve Logical Equivalence)**.** *If a formula ψ is obtained from a formula φ by finitely many applications of rules of replacement, then $\varphi \equiv \psi$.*
Go to proof.

Informal Proof Patterns

Definition 0.9.18 (Informal Proof Pattern)**.** An *informal proof pattern* is a reusable truth-preserving strategy for organizing an ordinary mathematical proof.

Semantic Justification of Argument Forms

Theorem 0.9.19 (Tautological Implication Gives a Valid Argument Form)**.** *Let $\Gamma = \{\gamma_1, \ldots, \gamma_n\} \subseteq \mathsf{WFF}$ and let $\varphi \in \mathsf{WFF}$. If*

$$(\gamma_1 \wedge \cdots \wedge \gamma_n) \to \varphi$$

is a tautology, then the argument form (Γ, φ) is valid.
Go to proof.

Argument Forms as a Bridge to Formal Systems

Summary and Transition

0.10 Compactness and Finite Satisfiability Preview

Orientation

Satisfiable Sets

Definition 0.10.1 (Satisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is satisfiable if there exists a truth assignment v such that $v \models \Gamma$.

$$\text{SatisfiableSet}(\Gamma) \iff \exists v \left(\text{TruthAssignment}(v) \wedge v \models \Gamma\right).$$

Definition 0.10.2 (Unsatisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is unsatisfiable if it is not satisfiable.

$$\text{UnsatisfiableSet}(\Gamma) \iff \neg\, \text{SatisfiableSet}(\Gamma).$$

Finitely Satisfiable Sets

Definition 0.10.3 (Finite Subset of a Set of Propositional Formulas). Let $\Gamma \subseteq \mathsf{WFF}$. A finite subset of Γ is a set $\Delta \subseteq \Gamma$ containing only finitely many formulas.

Definition 0.10.4 (Finitely Satisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is finitely satisfiable if every finite subset of Γ is satisfiable.

$$\text{FinitelySatisfiableSet}(\Gamma) \iff \forall \Delta \left((\Delta \subseteq \Gamma \wedge \Delta \text{ is finite}) \Rightarrow \text{SatisfiableSet}(\Delta) \right).$$

Finite Unsatisfiability Witnesses

Definition 0.10.5 (Unsatisfiability Witness). Let $\Gamma \subseteq \mathsf{WFF}$. A subset $\Delta \subseteq \Gamma$ is an unsatisfiability witness for Γ if Δ is unsatisfiable.

Definition 0.10.6 (Finite Unsatisfiability Witness). Let $\Gamma \subseteq \mathsf{WFF}$. A subset $\Delta \subseteq \Gamma$ is a finite unsatisfiability witness for Γ if Δ is finite and unsatisfiable.

$$\text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta).$$

Theorem 0.10.7 (Finite Unsatisfiability Witness). *Let $\Gamma \subseteq \mathsf{WFF}$. If there exists a finite $\Delta \subseteq \Gamma$ such that Δ is unsatisfiable, then Γ is unsatisfiable.*

Go to proof.

Corollary 0.10.8 (Finite Witness Contrapositive). *Let $\Gamma \subseteq \mathsf{WFF}$. If Γ is satisfiable, then every finite subset of Γ is satisfiable.*

Go to proof.

Propositional Compactness

Theorem 0.10.9 (Propositional Compactness). *Let $\Gamma \subseteq \mathsf{WFF}$. If every finite subset of Γ is satisfiable, then Γ is satisfiable.*

Go to proof.

Finite Truth Table Intuition

Summary

Proofs

Predicate Logic

0.11 Model-Theoretic View

0.12 Axiom Schemas

0.13 Syntax

0.13.1 Syntax of First-Order Logic: Terms and Formulas

Definition 0.13.1 (Variable)**.** For a first-order language $\mathcal{L}$, a *variable* is a symbol drawn from the distinguished variable stock of $\mathcal{L}$. Variables are usually written

$$x, y, z, x_0, x_1, x_2, \ldots.$$

Definition 0.13.2 (Constant Symbol)**.** For a first-order language $\mathcal{L}$, a *constant symbol* is a non-logical symbol of arity 0. A constant symbol may occur as a term without taking input terms.

Definition 0.13.3 (Function Symbol)**.** For a first-order language $\mathcal{L}$, an n-ary *function symbol* is a non-logical symbol of arity $n \geq 1$. If f has arity n, then it forms terms by the pattern

$$f(t_1, \ldots, t_n).$$

Definition 0.13.4 (Predicate Symbol)**.** For a first-order language $\mathcal{L}$, an n-ary *predicate symbol* is a non-logical symbol of arity $n \geq 1$. If P has arity n, then it forms atomic formulas by the pattern

$$P(t_1, \ldots, t_n).$$

Definition 0.13.5 (Term)**.** The set of *terms* of a first-order language $\mathcal{L}$, denoted $\mathsf{Term}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every variable is a term;
2. every constant symbol is a term;
3. if f is an n-ary function symbol and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term;
4. no string is a term unless it is generated by finitely many applications of the previous clauses.

Definition 0.13.6 (Atomic Formula)**.** An *atomic formula* of a first-order language $\mathcal{L}$ is a string of the form

$$P(t_1, \ldots, t_n),$$

where P is an n-ary predicate symbol and $t_1, \ldots, t_n$ are terms.

Definition 0.13.7 (Formula)**.** For a first-order language $\mathcal{L}$, a *formula* is an expression formed by the formula grammar of $\mathcal{L}$. The grammar starts from atomic formulas and closes under the logical connectives and quantifiers of the language.

Definition 0.13.8 (Well-Formed Formula)**.** The set of *well-formed formulas* of a first-order language $\mathcal{L}$, denoted $\mathsf{Form}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every atomic formula is a well-formed formula;
2. if φ is a well-formed formula, then $\neg\varphi$ is a well-formed formula;
3. if φ and ψ are well-formed formulas and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi)$ is a well-formed formula;
4. if φ is a well-formed formula and x is a variable, then $\forall x\, \varphi$ and $\exists x\, \varphi$ are well-formed formulas;
5. no string is a well-formed formula unless it is generated by finitely many applications of the previous clauses.

Definition 0.13.9 (Molecular Formula)**.** A *molecular formula* is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula whose final formation step uses a logical connective or a quantifier.

0.13.2 Syntax: Free Variables, Bound Variables, and Substitution

Definition 0.13.10 (Scope of a Quantifier)**.** Let φ be a formula of a first-order language.
If φ is of the form $\forall x\, \psi$ or $\exists x\, \psi$, then the formula ψ is called the *scope* of the quantifier.
The quantifier is said to *bind* all occurrences of the variable x that appear within its scope.

Definition 0.13.11 (Bound and Free Occurrences)**.** An occurrence of a variable x in a formula φ is *bound* if it lies within the scope of a quantifier $\forall x$ or $\exists x$.
An occurrence of x is *free* if it is not bound by any quantifier in φ.

Definition 0.13.12 (Free Variables of a Formula)**.** The set $\mathrm{FV}(\varphi)$ of *free variables* of a formula φ is defined recursively:

1. If $\varphi = P(t_1, \dots, t_n)$ is atomic, then $\mathrm{FV}(\varphi) = \bigcup_{i=1}^{n} \mathrm{Var}(t_i)$, where $\mathrm{Var}(t_i)$ is the set of variables in term t_i.
2. $\mathrm{FV}(\neg\varphi) = \mathrm{FV}(\varphi)$.
3. $\mathrm{FV}(\varphi \circ \psi) = \mathrm{FV}(\varphi) \cup \mathrm{FV}(\psi)$ for any binary connective $\circ$.
4. $\mathrm{FV}(\forall x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.
5. $\mathrm{FV}(\exists x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.

Definition 0.13.13 (Sentence)**.** A *sentence* (or *closed formula*) is a formula with no free variables.
Formally, φ is a sentence if and only if $\mathrm{FV}(\varphi) = \varnothing$.

Definition 0.13.14 (Substitution Notation)**.** Let φ be a formula, x a variable, and t a term.
The expression $\varphi[t/x]$ denotes the formula obtained from φ by replacing every *free* occurrence of x with the term t, leaving bound occurrences of x unchanged.

Definition 0.13.15 (Free for Substitution)**.** A term t is *free for* x in φ if no free occurrence of x in φ lies within the scope of a quantifier $\forall y$ or $\exists y$ where y is a variable occurring in t.
Equivalently, t is free for x in φ if the substitution $\varphi[t/x]$ does not result in any variable in t becoming bound.

Definition 0.13.16 (Alpha-Equivalence)**.** Formulas that differ only by the names of bound variables are logically equivalent. This is called *alpha-equivalence* (or *alphabetic variance*):

$$\forall x\, \varphi \;\equiv\; \forall y\, \varphi[y/x] \quad \text{(provided } y \text{ is not free in } \varphi\text{)}.$$

0.13.3 Syntax: Formula Depth

Definition 0.13.17 (Formula Depth)**.** The *depth* (or *complexity*) of a formula φ, denoted $\text{depth}(\varphi)$, is a natural number defined recursively:

1. **Atomic case.** If φ is atomic, then $\text{depth}(\varphi) = 0$.
2. **Negation.** If $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.
3. **Binary connectives.** If $\varphi = (\psi \circ \chi)$ for $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$, then
$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$
4. **Quantifiers.** If $\varphi = \forall x\, \psi$ or $\varphi = \exists x\, \psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.

0.14 Semantics

0.14.1 Semantics: Structures and Variable Assignments

Definition 0.14.1 (First-Order Language)**.** A *first-order language* $\mathcal{L}$ consists of variables, logical symbols, and a signature of non-logical symbols: constant symbols, function symbols with assigned arities, and predicate symbols with assigned arities. Equality may be included as a distinguished logical predicate.

Definition 0.14.2 (Structure)**.** Let $\mathcal{L}$ be a first-order language. A *structure* for $\mathcal{L}$ is a pair
$$\mathcal{M} = \langle D, I \rangle$$
where D is a nonempty set and I assigns each constant symbol an element of D, each n-ary function symbol a function $D^n \to D$, and each n-ary predicate symbol a relation on D^n.

Definition 0.14.3 (Variable Assignment)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure for $\mathcal{L}$. A *variable assignment* for $\mathcal{M}$ is a function
$$s : \mathsf{Var}_{\mathcal{L}} \to D.$$

Definition 0.14.4 (Modified Assignment)**.** Let $s : \mathsf{Var}_{\mathcal{L}} \to D$, let $x \in \mathsf{Var}_{\mathcal{L}}$, and let $d \in D$. The *modified assignment* $s[x \mapsto d]$ is the assignment defined by
$$s[x \mapsto d](y) = \begin{cases} d, & y = x, \\ s(y), & y \neq x. \end{cases}$$

Definition 0.14.5 (Term Evaluation)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure and s a variable assignment. The *value* of a term t, denoted $[\![t]\!]_{\mathcal{M},s}$, is defined recursively by
$$[\![x]\!]_{\mathcal{M},s} = s(x), \qquad [\![c]\!]_{\mathcal{M},s} = I(c),$$
and
$$[\![f(t_1, \ldots, t_n)]\!]_{\mathcal{M},s} = I(f)\big([\![t_1]\!]_{\mathcal{M},s}, \ldots, [\![t_n]\!]_{\mathcal{M},s}\big).$$

0.14.2 Semantics: Satisfaction and Truth

Definition 0.14.6 (Satisfaction)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure, let s be a variable assignment, and let φ be a well-formed formula. The relation $\mathcal{M}, s \models \varphi$ is defined recursively by the usual clauses: atomic formulas are evaluated by term values and interpreted predicate relations; equality compares term values; connectives follow the truth conditions for $\neg, \wedge, \vee, \to, \leftrightarrow$; and
$$\mathcal{M}, s \models \forall x\, \psi \iff \forall d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi),$$
$$\mathcal{M}, s \models \exists x\, \psi \iff \exists d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi).$$

Definition 0.14.7 (Truth in a Structure)**.** Let φ be a sentence and let $\mathcal{M}$ be a structure. The sentence φ is *true in* $\mathcal{M}$, written $\mathcal{M} \models \varphi$, exactly when $\mathcal{M}, s \models \varphi$ for every variable assignment s for $\mathcal{M}$.

Definition 0.14.8 (Validity)**.** A formula φ is *valid*, or *logically true*, exactly when $\mathcal{M}, s \models \varphi$ for every structure $\mathcal{M}$ for the language of φ and every variable assignment s for $\mathcal{M}$.

Definition 0.14.9 (Satisfiability)**.** A formula φ is *satisfiable* exactly when there are a structure $\mathcal{M}$ and a variable assignment s such that $\mathcal{M}, s \models \varphi$.

0.14.3 Semantics: Models, Theories, and Logical Consequence

Definition 0.14.10 (Model of a Sentence)**.** Let φ be a sentence and let $\mathcal{M}$ be a structure for the language of φ. The structure $\mathcal{M}$ is a *model of* φ exactly when $\mathcal{M} \models \varphi$.

Definition 0.14.11 (Model of a Set of Sentences)**.** Let T be a set of sentences and let $\mathcal{M}$ be a structure for their common language. The structure $\mathcal{M}$ is a *model of* T, written $\mathcal{M} \models T$, exactly when $\mathcal{M} \models \theta$ for every $\theta \in T$.

Definition 0.14.12 (Theory)**.** In Volume I, a *theory* is a set of sentences in a fixed first-order language.

Definition 0.14.13 (Logical Consequence)**.** Let T be a set of sentences and let φ be a sentence in the same language. The sentence φ is a *logical consequence* of T, written $T \models \varphi$, exactly when every model of T is a model of φ.

Definition 0.14.14 (Logical Equivalence)**.** Two sentences φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, exactly when each is a logical consequence of the other:

$$\varphi \models \psi \quad \text{and} \quad \psi \models \varphi.$$

0.14.4 Semantics: Open Formulas, Definable Relations, and Substitution Lemmas

Definition 0.14.15 (Open Formula)**.** An *open formula* is a well-formed formula with at least one free variable. When the free variables are among $x_1, \ldots, x_n$, it may be displayed as $\varphi(x_1, \ldots, x_n)$.

Definition 0.14.16 (Definable Relation)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure and let $\varphi(x_1, \ldots, x_n)$ be an open formula. The *relation defined by* φ in $\mathcal{M}$ is the set

$$R_\varphi^{\mathcal{M}} = \{(a_1, \ldots, a_n) \in D^n : \mathcal{M}, s[x_1 \mapsto a_1, \ldots, x_n \mapsto a_n] \models \varphi\}.$$

Lemma 0.14.17 (Substitution Lemma for Terms)**.** *Let $\mathcal{M}$ be a structure, let s be a variable assignment, let x be a variable, and let t and u be terms. Then*

$$[\![t[u/x]]\!]_{\mathcal{M},s} = [\![t]\!]_{\mathcal{M},\, s[x \mapsto [\![u]\!]_{\mathcal{M},s}]}.$$

Go to proof.

Lemma 0.14.18 (Substitution Lemma for Formulas)**.** *Let φ be a formula and let u be a term free for x in φ. For every structure $\mathcal{M}$ and assignment s,*

$$\mathcal{M}, s \models \varphi[u/x] \iff \mathcal{M}, s[x \mapsto [\![u]\!]_{\mathcal{M},s}] \models \varphi.$$

Go to proof.

0.15 Quantifiers

Basic Syntax and Semantics

Definition 0.15.1 (Universal Formula)**.** A *universal formula* is a well-formed formula of the form

$$\forall x\, \varphi,$$

where x is a variable and φ is a well-formed formula.

Definition 0.15.2 (Existential Formula)**.** An *existential formula* is a well-formed formula of the form

$$\exists x\, \varphi,$$

where x is a variable and φ is a well-formed formula.

Definition 0.15.3 (Bounded Quantifier)**.** A *bounded quantifier* is an abbreviation that restricts a quantifier to a specified set or predicate:

$$\begin{aligned} \forall x \in A\, \varphi &\quad :=\quad \forall x\, (x \in A \to \varphi), \\ \exists x \in A\, \varphi &\quad :=\quad \exists x\, (x \in A \land \varphi). \end{aligned}$$

Definition 0.15.4 (Sentence)**.** A *sentence* is a well-formed formula with no free variables.

Definition 0.15.5 (Open Formula)**.** An *open formula* is a well-formed formula with at least one free variable.

Definition 0.15.6 (Quantifier Scope)**.** In a formula $Qx\, \varphi$, where $Q \in \{\forall, \exists\}$, the *scope* of the quantifier Qx is the formula φ. Occurrences of x inside that scope are bound by the quantifier unless an inner quantifier for x intervenes.

Theorem 0.15.7 (Negation of Bounded Quantifiers)**.** *Let A be a set and let φ be a formula. Then*

$$\neg(\forall x \in A\, \varphi) \equiv \exists x \in A\, \neg\varphi, \qquad \neg(\exists x \in A\, \varphi) \equiv \forall x \in A\, \neg\varphi.$$

Go to proof.

Quantifier Laws

Theorem 0.15.8 (Quantifier Negation Laws)**.** *For every formula φ,*

$$\neg\forall x\, \varphi \equiv \exists x\, \neg\varphi, \qquad \neg\exists x\, \varphi \equiv \forall x\, \neg\varphi.$$

Go to proof.

Theorem 0.15.9 (Double Negation and Quantifiers)**.** *For every formula φ,*

$$\neg\neg\forall x\, \varphi \equiv \forall x\, \varphi, \qquad \neg\neg\exists x\, \varphi \equiv \exists x\, \varphi.$$

Go to proof.

Theorem 0.15.10 (Same-Type Quantifier Commutation)**.** *Quantifiers of the same type commute:*

$$\forall x\, \forall y\, \varphi \equiv \forall y\, \forall x\, \varphi, \qquad \exists x\, \exists y\, \varphi \equiv \exists y\, \exists x\, \varphi.$$

Go to proof.

Theorem 0.15.11 (Mixed Quantifiers Do Not Commute)**.** *In general,*

$$\forall x\, \exists y\, \varphi \not\equiv \exists y\, \forall x\, \varphi.$$

Go to proof.

Theorem 0.15.12 (Valid Quantifier Distribution). *For all formulas φ, ψ,*

$$\forall x\,(\varphi \wedge \psi) \equiv (\forall x\,\varphi) \wedge (\forall x\,\psi),$$

$$\exists x\,(\varphi \vee \psi) \equiv (\exists x\,\varphi) \vee (\exists x\,\psi).$$

Go to proof.

Theorem 0.15.13 (Invalid Quantifier Distribution). *In general,*

$$\forall x\,(\varphi \vee \psi) \not\equiv (\forall x\,\varphi) \vee (\forall x\,\psi),$$

$$\exists x\,(\varphi \wedge \psi) \not\equiv (\exists x\,\varphi) \wedge (\exists x\,\psi).$$

Go to proof.

Theorem 0.15.14 (Vacuous Quantification). *If x does not occur free in φ, then*

$$\forall x\,\varphi \equiv \varphi, \qquad \exists x\,\varphi \equiv \varphi.$$

Go to proof.

Theorem 0.15.15 (Renaming Bound Variables). *If y is free for x in φ and no capture is introduced, then*

$$\forall x\,\varphi \equiv \forall y\,\varphi[y/x], \qquad \exists x\,\varphi \equiv \exists y\,\varphi[y/x].$$

Go to proof.

Prenex Normal Form

Definition 0.15.16 (Quantifier Prefix). A *quantifier prefix* is a finite string

$$Q_1x_1\,Q_2x_2 \cdots Q_nx_n,$$

where each $Q_i \in \{\forall, \exists\}$ and each x_i is a variable.

Definition 0.15.17 (Matrix). In a formula $Q_1x_1 \cdots Q_nx_n\,\psi$, the *matrix* is the trailing formula ψ. For prenex normal form, the matrix is required to be quantifier-free.

Definition 0.15.18 (Prenex Formula). A *prenex formula* is a formula of the form

$$Q_1x_1\,Q_2x_2 \cdots Q_nx_n\,\psi,$$

where $Q_1x_1 \cdots Q_nx_n$ is a quantifier prefix and ψ is quantifier-free.

Definition 0.15.19 (Prenex Normal Form). A formula ψ is a *prenex normal form* of a formula φ exactly when ψ is prenex and $\varphi \equiv \psi$.

Theorem 0.15.20 (Existence of Prenex Normal Form). *Every first-order formula is logically equivalent to some formula in prenex normal form.* *Go to proof.*

Logical Strength and Quantifier Order

Definition 0.15.21 (Stronger Formula). A sentence A is *stronger than* a sentence B exactly when

$$A \models B \quad \text{and} \quad B \not\models A.$$

Definition 0.15.22 (Weaker Formula). A sentence B is *weaker than* a sentence A exactly when A is stronger than B.

Definition 0.15.23 (Quantifier Order)**.** The *quantifier order* of a prenex formula is the ordered list of quantifiers in its prefix. A change of order is significant when it changes which witnesses may depend on earlier variables.

Theorem 0.15.24 (Universal Implies Existential)**.** *Over nonempty domains,*

$$\forall x\, \varphi(x) \models \exists x\, \varphi(x).$$

Go to proof.

Theorem 0.15.25 (Existential Does Not Imply Universal)**.** *In general,*

$$\exists x\, \varphi(x) \not\models \forall x\, \varphi(x).$$

Go to proof.

Theorem 0.15.26 (Uniform Witness Implies Dependent Witness)**.** *For every formula $\varphi(x, y)$,*

$$\exists y\, \forall x\, \varphi(x, y) \models \forall x\, \exists y\, \varphi(x, y).$$

Go to proof.

Theorem 0.15.27 (Dependent Witness Need Not Be Uniform)**.** *In general,*

$$\forall x\, \exists y\, \varphi(x, y) \not\models \exists y\, \forall x\, \varphi(x, y).$$

Go to proof.

0.16 Proof Theory

0.16.1 Proof Theory: Quantifier Inference Rules

Definition 0.16.1 (Universal Instantiation — UI)**.** From a universally quantified statement, infer any instance obtained by substituting a term for the bound variable:

$$\forall x\, \varphi \;\Rightarrow\; \varphi[t/x].$$

Definition 0.16.2 (Universal Generalization — UG)**.** From a formula, infer a universally quantified statement:

$$\varphi \;\Rightarrow\; \forall x\, \varphi.$$

Definition 0.16.3 (Existential Instantiation — EI)**.** From an existentially quantified statement, infer an instance with a fresh constant (witness):

$$\exists x\, \varphi \;\Rightarrow\; \varphi[c/x].$$

Definition 0.16.4 (Existential Generalization — EG)**.** From a formula containing a term, infer an existential statement:

$$\varphi[t/x] \;\Rightarrow\; \exists x\, \varphi.$$

0.16.2 Proof Strategies for Quantified Statements

Strategy 1: Proving a Universal $\forall x\, \varphi(x)$

Strategy 2: Proving an Existential $\exists x\, \varphi(x)$

Strategy 3: Using a Universal $\forall x\, \varphi(x)$

Strategy 4: Using an Existential $\exists x\, \varphi(x)$

The Central Asymmetry: Universal vs. Existential Goals

Nested Quantifiers: The $\forall\exists$ Pattern in Analysis

A Complete Worked Example

0.16.3 Proof Theory: Equality Rules

Definition 0.16.5 (Equality Introduction — Reflexivity)**.** For any term t, one may infer $t = t$.

Definition 0.16.6 (Equality Elimination — Substitution of Equals)**.** From $t_1 = t_2$ and $\varphi[t_1/x]$, one may infer $\varphi[t_2/x]$.

Theorem 0.16.7 (Symmetry of Equality)**.** *From $t_1 = t_2$, one may infer $t_2 = t_1$. Go to proof.*

Theorem 0.16.8 (Transitivity of Equality)**.** *From $t_1 = t_2$ and $t_2 = t_3$, one may infer $t_1 = t_3$. Go to proof.*

Theorem 0.16.9 (Term Substitution under Equality)**.** *If $t_1 = t_2$, then for any function symbol f, $f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$. Go to proof.*

Theorem 0.16.10 (Predicate Substitution under Equality)**.** *If $t_1 = t_2$ and P is an n-ary predicate symbol, then $P(\dots, t_1, \dots) \Leftrightarrow P(\dots, t_2, \dots)$. Go to proof.*

0.16.4 Proof Theory: Soundness and Completeness

Definition 0.16.11 (Soundness)**.** A proof system is *sound* if every provable formula is valid:

$$\Gamma \vdash \varphi \implies \Gamma \models \varphi.$$

Definition 0.16.12 (Completeness)**.** A proof system is *complete* if every valid formula is provable:

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Theorem 0.16.13 (Soundness of First-Order Logic)**.** *The standard inference rules for first-order logic (UI, UG, EI, EG, and the propositional rules) are sound: they preserve truth in all structures.*
If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$. Go to proof.

0.17 Translation

English to Logic and Scope Ambiguity

Definition 0.17.1 (Translation Key)**.** A *translation key* is a finite record that fixes a domain of discourse and assigns intended English readings to the constant symbols, predicate symbols, and relation symbols used in a translation.

Definition 0.17.2 (Domain of Discourse)**.** The *domain of discourse* for a translation is the collection of objects over which the variables of the translated formulas range.

Definition 0.17.3 (Predicate and Relation Symbols in a Translation)**.** In a translation key, a unary predicate symbol names a property of one object, and an n-ary relation symbol names a relation among n objects.

Definition 0.17.4 (Constants and Variables in a Translation)**.** A *constant* names a fixed object from the translation key, while a *variable* marks a place bound by a quantifier or left open for later binding.

Proposition 0.17.5 (Universal Translation Pattern)**.** *In a broad domain, an English statement of the form "All A are B" is translated by*

$$\forall x\,(A(x) \to B(x)).$$

Go to proof.

Proposition 0.17.6 (Existential Translation Pattern)**.** *In a broad domain, an English statement of the form "Some A are B" is translated by*

$$\exists x\,(A(x) \wedge B(x)).$$

Go to proof.

Proposition 0.17.7 (Conditional Translation Pattern)**.** *An English statement of the form "If A, then B" is translated by*

$$A \to B,$$

with quantifiers placed according to the variables occurring in A and B.
Go to proof.

Proposition 0.17.8 (Negation Translation Pattern)**.** *The negation of a translated quantified statement is obtained by negating the whole formula and then pushing the negation inward using the quantifier negation laws.*
Go to proof.

Definition 0.17.9 (Scope Ambiguity)**.** A sentence has *scope ambiguity* when the same ordinary-language wording admits two or more translations with different quantifier nesting or different connective scope.

Square of Opposition

Definition 0.17.10 (Universal Affirmative Form)**.** The *universal affirmative* categorical form, called A, translates "All A are B" as

$$\forall x\,(A(x) \to B(x)).$$

Definition 0.17.11 (Universal Negative Form)**.** The *universal negative* categorical form, called E, translates "No A are B" as

$$\forall x\,(A(x) \to \neg B(x)).$$

Definition 0.17.12 (Particular Affirmative Form)**.** The *particular affirmative* categorical form, called I, translates "Some A are B" as

$$\exists x\,(A(x) \wedge B(x)).$$

Definition 0.17.13 (Particular Negative Form)**.** The *particular negative* categorical form, called O, translates "Some A are not B" as

$$\exists x\,(A(x) \wedge \neg B(x)).$$

Proposition 0.17.14 (A-O Contradiction)**.** *The A-form $\forall x\,(A(x) \to B(x))$ and the O-form $\exists x\,(A(x) \wedge \neg B(x))$ are contradictory: exactly one of them is true.*
Go to proof.

Proposition 0.17.15 (E-I Contradiction)**.** *The E-form $\forall x\,(A(x) \to \neg B(x))$ and the I-form $\exists x\,(A(x) \wedge B(x))$ are contradictory: exactly one of them is true.*
Go to proof.

Proposition 0.17.16 (Contrariety Requires Existential Import)**.** *If $\exists x\,A(x)$, then the A-form and E-form cannot both be true.*
Go to proof.

0.18 Reference

Common Errors and Fallacies

Summary Tables

Proofs

Many-Sorted Logic

0.19 Introduction

0.19.1 Examples

0.19.2 Reduction to and comparison with first-order logic

0.19.3 Uses of many-sorted logic

0.19.4 Many-sorted logic as a unifier logic

0.20 Structures

0.20.1 Definition (signature)

0.20.2 Definition (structure)

0.21 Formal many-sorted language

0.21.1 Alphabet

0.21.2 Expressions: formulas and terms

0.21.3 Remarks on notation

0.21.4 Abbreviations

0.21.5 Induction

0.21.6 Free and bound variables

0.22 Semantics

0.22.1 Definitions

0.22.2 Satisfiability, validity, consequence and logical equivalence

0.23 Substitution of a term for a variable

0.24 Semantic theorems

0.24.1 Coincidence lemma

0.24.2 Substitution lemma

0.24.3 Equals substitution lemma

0.24.4 Isomorphism theorem

Standard Second-Order Logic

0.27 Introduction

0.27.1 General idea

0.27.2 Expressive power

0.27.3 Model-theoretic counterparts of expressiveness

0.27.4 Incompleteness

0.28 Second-order grammar

0.28.1 Definition (signature and alphabet)

0.28.2 Expressions: terms, predicates and formulas

0.28.3 Remarks on notation

0.28.4 Induction

0.28.5 Free and bound variables

0.28.6 Substitution

0.29 Standard structures

0.29.1 Definition of standard structures

0.29.2 Relations between standard structures established without the formal language

0.30 Standard semantics

0.30.1 Assignment

0.30.2 Interpretation

0.30.3 Consequence and validity

0.30.4 Logical equivalence

0.30.5 Simplifying our language

0.30.6 Alternative presentation of standard semantics

0.30.7 Definable sets and relations in a given structure

0.30.8 More about the expressive power of standard SOL

Languages, Axioms, and Models

Languages, Axioms, and Models Toolkit - Planned

0.32 Signatures

Signatures

Definition 0.32.1 (First-Order Signature)**.** A *first-order signature* is a tuple

$$\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar}),$$

where $\mathcal{C}$ is a set of constant symbols, $\mathcal{F}$ is a set of function symbols, $\mathcal{R}$ is a set of relation symbols, and

$$\mathrm{ar} : \mathcal{F} \cup \mathcal{R} \to \mathbb{Z}_{>0}$$

assigns to each function or relation symbol its number of arguments.

Definition 0.32.2 (Arity)**.** Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar})$ be a first-order signature. The *arity function* is the map

$$\mathrm{ar} : \mathcal{F} \cup \mathcal{R} \to \mathbb{Z}_{>0}$$

that assigns to each function or relation symbol the number of arguments it requires. When constants are treated as 0-ary function symbols, the codomain is enlarged to include 0.

Definition 0.32.3 (Language)**.** A *first-order language* has two standard readings. As vocabulary, it is the same data as a signature,

$$\mathcal{L} = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar}).$$

As a formal language generated by a signature Σ, it is the set of well-formed strings built from the alphabet

$$A = \Sigma \cup V \cup \Lambda,$$

where V is a countable set of variables and Λ is the fixed stock of logical symbols and punctuation. In this second reading,

$$\mathcal{L}(\Sigma) = \{ s \in A^* \mid s \in \mathrm{Terms}(\Sigma) \text{ or } s \in \mathrm{WFF}(\Sigma) \}.$$

Definition 0.32.4 (Constant Symbol)**.** A *constant symbol* is a non-logical symbol that behaves syntactically as a 0-ary function symbol. In the streamlined convention where constants are included among function symbols, this is the condition

$$\mathrm{Constant}(c) \iff c \in \mathcal{F} \text{ and } \mathrm{ar}(c) = 0.$$

Because its arity is 0, the symbol c is already a well-formed term and takes no input terms.

Definition 0.32.5 (Function Symbol)**.** A *function symbol* is a non-logical symbol $f \in \mathcal{F}$ whose assigned arity determines how many terms it must take as inputs. If $\mathrm{ar}(f) = n$, then f is an n-ary term constructor: it takes exactly n terms and produces a new term, not a truth value.

Definition 0.32.6 (Relation Symbol)**.** A *relation symbol*, or predicate symbol, is a non-logical symbol $R \in \mathcal{R}$ whose assigned arity determines how many terms it must take as inputs. If $\mathrm{ar}(R) = n$, then R is an n-ary formula constructor: it takes exactly n terms and produces an atomic formula, hence something with a truth value rather than an object.

0.33 Axioms

Axioms

Definition 0.33.1 (Axiom)**.** Let $\mathcal{L}$ be a first-order language, and let $\mathrm{Sent}(\mathcal{L})$ denote the set of sentences of $\mathcal{L}$. If $T \subseteq \mathrm{Sent}(\mathcal{L})$ is a theory, then an *axiom of* T is an element $\alpha \in T$.

Definition 0.33.2 (Axiom System)**.** An *axiom system* is a collection of axioms expressed in a common language.

Definition 0.33.3 (Assumption)**.** An *assumption* is a statement temporarily accepted for purposes of reasoning.

0.34 Theories

Theories

Definition 0.34.1 (Theory)**.** A *theory* over a first-order language $\mathcal{L}$ is a collection of closed sentences in that language:

$$T \subseteq \mathrm{Sentences}(\mathcal{L}).$$

If Γ is a chosen set of axioms, the theory generated by Γ is the deductive closure

$$T = \{\varphi \in \mathrm{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

Definition 0.34.2 (Theorem)**.** A *theorem* of a theory T is a sentence φ for which there is a finite formal derivation from T, written

$$T \vdash \varphi.$$

Equivalently, in proof-theoretic terms, φ is a theorem of T exactly when some derivation has leaves drawn from logical axioms or sentences of T, uses valid inference rules, and has φ as its root.

Definition 0.34.3 (Consequence)**.** A sentence φ is a *semantic consequence* of a set of premises Γ, written $\Gamma \vDash \varphi$, if every model of Γ is also a model of φ. Equivalently, there is no structure in which all sentences of Γ are true while φ is false.

Definition 0.34.4 (Deductive Closure)**.** Let $\Gamma \subseteq \mathrm{Sentences}(\mathcal{L})$ be a set of premises. The *deductive closure* of Γ, denoted $\mathrm{Cn}(\Gamma)$, is the set of all sentences derivable from Γ:

$$\mathrm{Cn}(\Gamma) = \{\varphi \in \mathrm{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

0.35 Models

Models

Definition 0.35.1 (Interpretation)**.** Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar})$ be a first-order signature, and let M be a nonempty set. An *interpretation of* Σ *on* M is an assignment $\mathcal{I}$ that maps each symbol of Σ to concrete data over M: constants are assigned elements of M, function symbols are assigned operations on M, and relation symbols are assigned relations on M.

Definition 0.35.2 ($\mathcal{L}$-Structure)**.** Let $\mathcal{L}$ be a first-order signature. An $\mathcal{L}$*-structure* is a pair

$$\mathfrak{M} = (M, \mathcal{I}),$$

where M is a nonempty domain and $\mathcal{I}$ is an interpretation of the symbols of $\mathcal{L}$ on M.

Definition 0.35.3 (Model)**.** Let $\mathcal{L}$ be a first-order signature, let T be a theory over $\mathcal{L}$, and let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$-structure. The structure $\mathfrak{M}$ is a *model of* T, written $\mathfrak{M} \vDash T$, if every sentence in T is true in $\mathfrak{M}$.

Definition 0.35.4 (Satisfaction)**.** Let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$-structure. A *variable assignment* is a function $s : \mathrm{Var} \to M$. The assignment extends recursively to a term-evaluation map $\bar{s}$ by

$$\bar{s}(x) = s(x), \qquad \bar{s}(c) = \mathcal{I}(c),$$

and

$$\bar{s}\big(f(t_1, \ldots, t_n)\big) = \mathcal{I}(f)\big(\bar{s}(t_1), \ldots, \bar{s}(t_n)\big).$$

The satisfaction relation $\mathfrak{M} \vDash \varphi[s]$ is then defined by induction on formulas: atomic relations are tested inside the interpreted relation, equality is interpreted as equality in M, Boolean connectives are interpreted by the corresponding truth operations, and quantifiers range over all elements of M.

Definition 0.35.5 (Countermodel)**.** Let $\Gamma \subseteq \mathrm{Sent}(\mathcal{L})$ be a set of premises, let $\varphi \in \mathrm{Sent}(\mathcal{L})$, and let $\mathfrak{M}$ be an $\mathcal{L}$-structure. A *countermodel to the entailment* $\Gamma \vDash \varphi$ is a structure $\mathfrak{M}$ that satisfies every sentence in Γ but does not satisfy φ.

0.36 Consistency

Consistency

Definition 0.36.1 (Contradiction)**.** A *contradiction* is a statement that cannot be true under a given interpretation.

Definition 0.36.2 (Consistent Axiom System)**.** A *consistent axiom system* is an axiom system that does not lead to contradiction.

Definition 0.36.3 (Inconsistent Axiom System)**.** An *inconsistent axiom system* is an axiom system containing or producing contradictory assertions.

Definition 0.36.4 (Model Existence Criterion (Informal))**.** Informally, if an axiom system has a model, then the axioms are mutually compatible.

Definition 0.36.5 (Relative Consistency)**.** Let T_0 and T_1 be theories. The theory T_1 is *consistent relative to* T_0 if

$$\mathrm{Con}(T_0) \implies \mathrm{Con}(T_1).$$

Equivalently, every contradiction in T_1 would imply a contradiction in T_0.

0.37 Independence

Independence Toolkit - Planned

0.38 Comparing Theories

Comparing Theories Toolkit - Planned

0.39 Examples And Previews

Examples And Previews Toolkit - Planned

0.40 Summary

Summary Toolkit - Planned

Proofs